

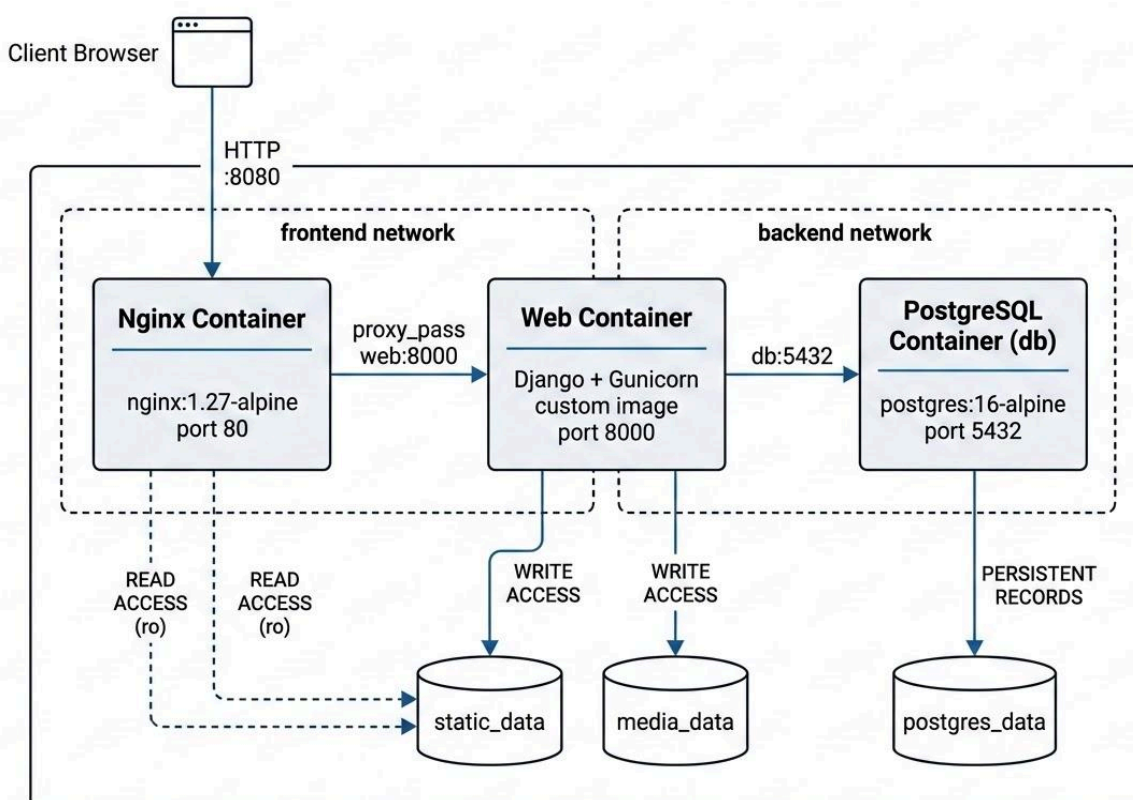
Documentação técnica: Django containerizado com Docker Compose

Repositório: <https://github.com/rafantavares/atv-docker-compose>

Nossa aplicação é um sistema simples de upload de arquivos em Django. Ela roda em 3 containers orquestrados pelo Docker Compose: Nginx, Django/Gunicorn e PostgreSQL.

1. Visão geral da arquitetura

O cliente só acessa o Nginx, pela porta 8080. O Nginx repassa as páginas para o Gunicorn, que executa o Django. O Django grava os dados no PostgreSQL e os arquivos enviados em um volume.



Componente	Porta	Redes	Volumes
nginx	8080 (host) → 80 (container)	frontend	media_data, static_data (somente leitura)
web	8000 (só interna)	frontend, backend	media_data, static_data
db	5432 (só interna)	backend	postgres_data

2. Imagem Docker da aplicação Django

Arquivo: Dockerfile

Decisão	O que fizemos	Por quê
Imagem base	python:3.12-slim	Imagem oficial, pequena e já vem com Python e pip. A slim é bem menor que a imagem completa. Não usamos a alpine porque ela usa outra biblioteca C (musl) e às vezes precisa compilar pacotes Python.
Variáveis	PYTHONUNBUFFERED=1, PYTHONDONTWRITEBYTE CODE=1, PIP_NO_CACHE_DIR=1	Os logs aparecem na hora no docker compose logs, o Python não gera .pyc e a imagem fica menor.
WORKDIR	/app	Todos os comandos e o código ficam nessa pasta.
Dependências	COPY requirements.txt e depois RUN pip install	Copiamos as dependências antes do código. Assim o Docker reaproveita essa camada do cache quando só o código muda, e o build fica rápido.
Arquivos	COPY app/ . e COPY entrypoint.sh /entrypoint.sh	Dentro da imagem fica /app/manage.py, /app/config/ (settings, urls, wsgi) e /app/arquivos/ (o app de upload).
Build	sed + chmod + useradd + mkdir + chown	O sed remove a quebra de linha do Windows do script e o chmod deixa ele executável. Também criamos o usuário appuser e as

Decisão	O que fizemos	Por quê
		pastas media e staticfiles com o dono certo, para ele conseguir gravar nos volumes.
Usuário	USER appuser	A aplicação não roda como root, o que é mais seguro.
Inicialização	ENTRYPOINT ["/entrypoint.sh"]	Antes de subir, roda migrate (cria as tabelas) e collectstatic (junta o CSS/JS). Depois chama o Gunicorn com exec.
Servidor	CMD ["gunicorn", "config.wsgi:application", "--bind", "0.0.0.0:8000", "--workers", "3"]	O runserver do Django é só para desenvolvimento, então usamos o Gunicorn (servidor WSGI). O 0.0.0.0 permite que o Nginx acesse o container e os 3 workers atendem requisições em paralelo.

3. Orquestração com Docker Compose

Arquivo: docker-compose.yml. Ele define 3 serviços, 3 volumes nomeados e 2 redes.

	db	web	nginx
Responsabilidade	Banco de dados	Aplicação Django	Proxy reverso e arquivos
Imagem	postgres:16-alpine	própria (build: . → django-upload:1.0)	nginx:1.27-alpine
Portas	nenhuma publicada (5432 interna)	expose: 8000 (só interna)	8080:80
Variáveis	POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD (vindas do .env)	env_file: .env (chave, debug, hosts e dados do banco)	nenhuma
Volumes	postgres_data	media_data, static_data	default.conf (bind), media_data:ro, static_data:ro
Redes	backend	backend, frontend	frontend

	db	web	nginx
Dependência	nenhuma	db com condition: service_healthy	web
Inicialização	restart: unless-stopped + healthcheck pg_isready	restart: unless-stopped	restart: unless-stopped

O healthcheck faz o web só subir quando o Postgres já aceita conexões. Sem ele, o migrate poderia falhar.

4. Comunicação entre os containers

- O Compose cria as redes frontend e backend do tipo **bridge**. Nelas, o Docker tem um **DNS interno** que resolve o **nome do serviço** para o IP do container. Por isso usamos nomes, e não IPs, que mudam a cada recriação.
- **Nginx → Django:** pela rede frontend. O Nginx chama web:8000 (upstream django { server web:8000; }), e o Gunicorn escuta em 0.0.0.0:8000.
- **Django → PostgreSQL:** pela rede backend. No settings.py, HOST vem de POSTGRES_HOST=db e a porta é a 5432.
- **Portas internas × externas:** só a 8080 é publicada no host (ports). A 8000 e a 5432 existem apenas dentro das redes Docker. Ninguém de fora acessa o Gunicorn ou o banco direto.
- **Isolamento:** o web é o único que está nas duas redes. O Nginx não enxerga o banco.

5. Nginx e proxy reverso

O Nginx é a única porta de entrada. Ele recebe as requisições na porta 80 do container (8080 no host). As páginas vão para o Django; os arquivos (/media/ e /static/) ele mesmo entrega, lendo dos volumes.

Arquivo: nginx/default.conf

```
upstream django {
    server web:8000;           # nome do serviço + porta interna do
                                # Gunicorn
}
server {
    listen 80;                 # porta do container (8080 no host)
    client_max_body_size 20M;  # limite de upload (o padrão do Nginx
                                # é 1 MB)

    location /static/ { alias /app/staticfiles/; } # direto do volume
static_data
    location /media/ { alias /app/media/; }        # direto do volume
media_data

    location / {
        proxy_pass http://django;                 # aqui acontece o
proxy reverso
        proxy_set_header Host $http_host;          # mantém
localhost:8080 (CSRF/ALLOWED_HOSTS)
        proxy_set_header X-Real-IP $remote_addr;   # IP real do
cliente
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Portas envolvidas: cliente → 8080 (host) → 80 (Nginx) → 8000 (Gunicorn).

6. Persistência de dados

Container é descartável: tudo que ele grava na própria camada some quando ele é removido. Por isso usamos volumes nomeados, que existem fora do container.

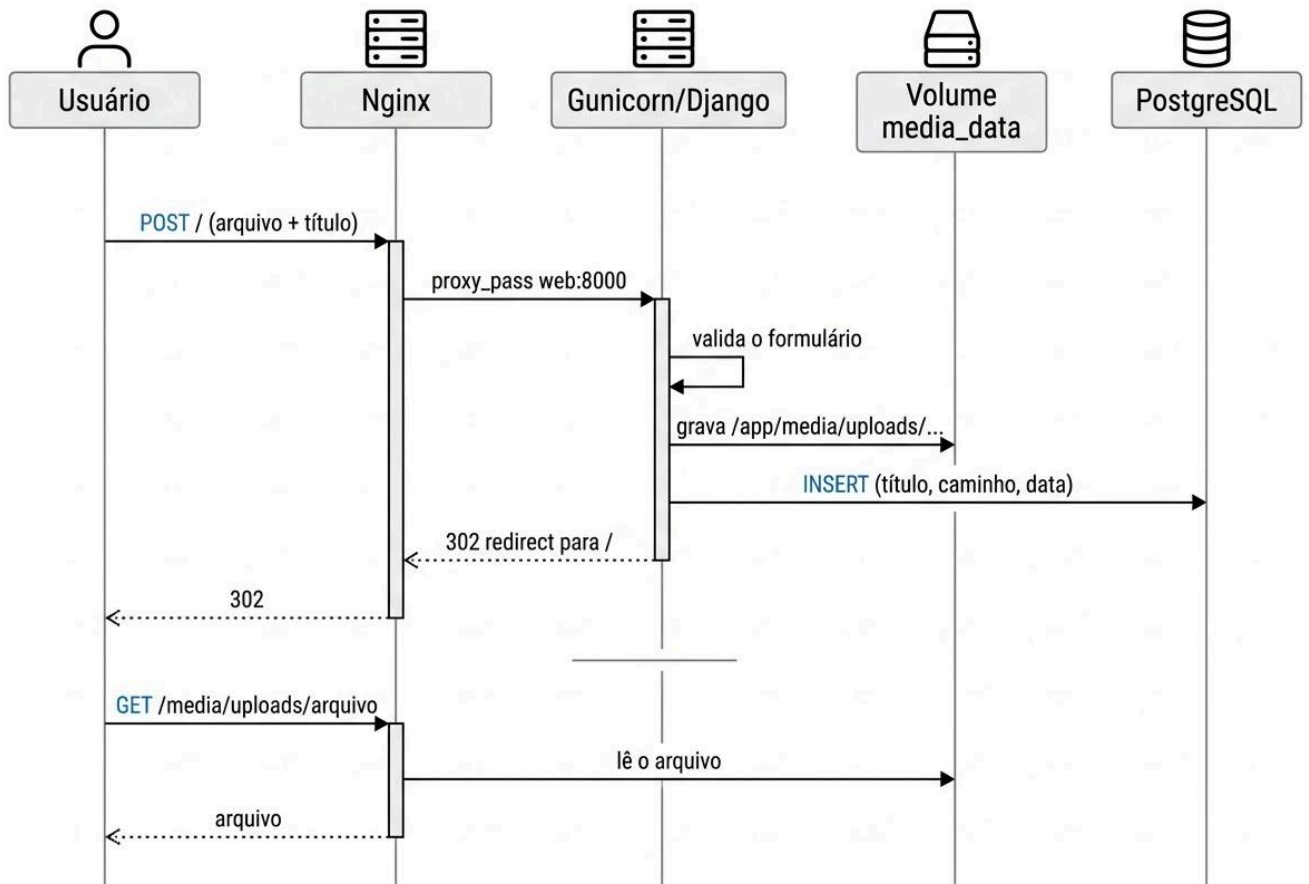
Dado	Volume	Montado em	O que acontece se remover e recriar os containers
Banco PostgreSQL	postgres_data	db:/var/lib/postgresql/data	O container novo monta o mesmo volume, e as tabelas e registros continuam.
Arquivos enviados	media_data	web:/app/media e nginx:/app/media	Os arquivos continuam no volume, e os links voltam a funcionar.
Arquivos estáticos	static_data	web:/app/staticfiles e nginx:/app/staticfiles	São gerados de novo pelo collectstatic a cada subida.

- docker compose down: remove os containers e as redes, mas mantém os volumes.
- docker compose down -v: remove também os volumes, e aí os dados são perdidos.

Testamos isso: enviamos um arquivo, rodamos down e depois up, e o arquivo continuou listado e abrindo.

7. Fluxo do upload de um arquivo

1. O usuário escolhe o arquivo e envia o formulário (POST /, multipart/form-data, com token CSRF).
2. O Nginx recebe na porta 80 e verifica o tamanho (até 20 MB). Depois repassa para web:8000.
3. O Gunicorn entrega a requisição ao Django, e a view index valida o ArquivoForm.
4. No form.save(), o FileField grava o arquivo em /app/media/uploads/AAAA/MM/DD/, que é o volume media_data.
5. O Django grava no PostgreSQL (volume postgres_data) só o título, o caminho e a data, não o arquivo.
6. O Django redireciona para /. A lista mostra o link /media/uploads/..., que o Nginx entrega direto do volume.



8. Análise técnica

Vantagens

1. O ambiente inteiro sobe com um comando (docker compose up) e roda igual em qualquer máquina com Docker.
2. Os dados e os uploads ficam em volumes, então sobrevivem à remoção dos containers.
3. Há isolamento e separação de papéis: só a porta 8080 fica exposta, o banco fica numa rede que o Nginx não acessa, e o Nginx entrega os arquivos sem ocupar o Django.

Limitações

1. Não tem HTTPS: tudo trafega em HTTP.
2. Tudo roda em um único host. Se a máquina cair, a aplicação cai, e os volumes locais não têm backup.
3. Qualquer pessoa que tenha o link acessa o arquivo em /media/, e não há validação de tipo de arquivo nem autenticação para enviar.

Melhorias para produção

1. Colocar HTTPS no Nginx (certificado Let's Encrypt) e ativar as configurações de segurança do Django.
2. Guardar os uploads em um armazenamento de objetos (ex.: S3) e fazer backup do banco (pg_dump agendado).
3. Usar secrets para as senhas em vez do .env, colocar um healthcheck no web e limites de CPU/memória, e montar um pipeline de CI/CD para build e deploy.